循环神经网络：时序参数共享、双向结构与递归网络

读前须知

- **前置**：前馈网络、反向传播；对“参数共享”有直觉会更好理解。
- **这一篇的位置**：序列建模的起点。它让同一个网络在不同时间步上重复使用，从而处理长短不一的输入。
- **读完能做什么**：写出 RNN 的隐藏状态更新式；解释沿时间反向传播（BPTT）为什么会梯度爆炸或消失；说清双向结构与递归网络的差别。

一、序列数据与两类 RNN

1.1 从一本诗集说起

2017 年，一本名为《阳光失了玻璃窗》的诗集出版。书本身平平无奇，特别之处在于作者是微软小冰——一个聊天机器人。诗集背后是循环神经网络一类的序列模型，它们把词语按概率上的先后关系排列组合，拼出的句子读起来竟也像模像样。

抛开作品质量不谈，这类模型的价值在于处理了一类此前难以处理的数据：**有先后顺序的数据。**图像、表格、静态的样本，各维度之间没有天然的次序；而语言、语音、股价、心电图，顺序本身就是信息的一部分，调换顺序，含义就变了。

1.2 循环与递归

在深度学习中，RNN 这个缩写有两层含义：既可以是**循环神经网络**（recurrent neural network），也可以是**递归神经网络**（recursive neural network）。

两者关系密切：循环神经网络可以看成递归神经网络的一个特例——把递归结构限制成一条链；递归神经网络则可以视为循环神经网络的推广——把链换成任意树结构。

本篇先讲循环网络，再看它的树状推广。

二、时间维度的参数共享

2.1 变长输入带来的困难

循环神经网络与前面介绍的所有网络都不同，它的独特之处在于引入了**时间**这个维度。

回想 029 中的卷积网络：它在空间上共享参数，同一个核函数可以扫过图像的不同区域。把这种共享搬到时间维度上，让网络用相同的权重系数来处理具有先后顺序的数据，得到的就是循环神经网络。

这个迁移之所以必要，是因为输入长度可变。定义在空间上的数据不会无限延伸，再长的画卷也有确定的边界，图像识别任务中输入像素的数目往往是固定的。但以时间为自变量的序列，很难说清终点在哪里，有时终点根本不存在——一段对话、一路行情，都可以无限延续下去。

如果对每个时间点都单独配置一套权重，参数会随序列长度线性膨胀，而且训练时从未见过的长度根本无法处理。循环神经网络的做法是把长度不定的输入切分成等长的小块，用同一组权重逐块处理，从而以固定数量的参数应对任意长度的输入。

2.2 共享的代价与收益

参数共享带来的收益是双重的：参数少，训练所需的样本就少；同一套规则在所有时刻复用，模型天然能处理训练时没见过的序列长度。

代价则是表达能力受到约束——所有时刻被迫使用同一套变换。但这恰恰是想要的：序列数据的规律本来就是不随时间变化的，“主语之后常跟谓语”这条规则，在句首成立，在句尾也成立。

补充说明：从计算图的角度看，循环网络并不是“有环”的网络。训练时它会沿时间轴展开成一个极深的前馈网络，每个时刻是一层，层与层之间共享同一组参数。所以反向传播、梯度消失这些概念都可以直接套用，只是“层数”变成了“时间步数”。理解这一点，后面讲 BPTT 和梯度弥散时会顺畅很多。

三、记忆机制

3.1 时域关联

时间维度的参数共享，功能上是为了利用数据之间的**时域关联性**。

生活中这样的经验比比皆是：前后文的语境能够对信息做有效补充。英语考试里的完形填空就是典型——根据上下文语义选出合适的词。又如妈妈在厨房喊一声“菜炒好了，赶紧来……”，后面的话没听清，也能猜到十有八九是叫吃饭，而不是叫洗衣服。

循环神经网络对时域关联的利用，体现在时刻 t 的输出既取决于当前输入，也取决于前一时刻 t−1 乃至更早的输出。这在结构上引入了**反馈**，因而网络具有了**记忆**。

3.2 两种记忆

严格说来，前馈网络也有某种意义上的记忆：参数经过最优化后，优化的结果中就包含着过往数据的痕迹。但这种记忆只存在于训练集上——模型一旦训练完成，参数就冻结了，应用到新数据时不会随输入而改变。这是一种**冻结的记忆**。

循环网络的记忆则不同。它的作用不是给每种特征分配固定权重，而是描述一系列时序事件之间的关系。即便这些事件之间没有明显而紧密的时间关联，它们之间的相关性依然可能如草蛇灰线般伏延千里，而这正是循环网络的记忆要挖掘的对象。

3.3 两种知识

两种网络对应两种知识类型。

前馈网络适合表示**客观性知识**。学会分辨颜色之后，这项技能会伴随一生，不随时间或环境改变——一块红布不会因为放了几十年、或者拿到另一座城市就变成蓝色。这类知识的规律是静态的。

循环网络适合表示**主观性知识**。每种语言中都有同音不同义的词，听到一个“jing”的音节，要根据前后的音节判断它是“干净”的净还是“安静”的静。这类知识藏在数据的顺序里，而顺序恰好可以由循环网络刻画。

> **新手提示：隐藏状态不是输出**
>
> 初学时常把隐藏状态 h 与输出 y 混为一谈。h 是网络的内部记忆，只在网络内部流转，不对外暴露；y 才是这一时刻对外给出的结果。二者由不同的权重矩阵产生：h 由输入权重 W 与状态转移权重 U 算出，y 通常再由 h 经输出权重 V 变换得到。多对一的任务（如情感分类）只取最后一个时刻的 y，一对多的任务（如图像描述）则把同一个 h 反复喂给解码端。

四、结构与前向计算

4.1 状态更新式

输入序列的内部信息存储在隐藏层中，并随时间在隐藏层里流转。这个过程可以写成一个递推式：

$$\mathbf{h}_t = f\left(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1}\right)$$

含义是：把时刻 t 的输入 x_t 的加权结果，与时刻 t−1 的隐藏状态 h_{t−1} 的加权结果相加，一起送进传递函数 f，得到时刻 t 的隐藏状态 h_t。

W 是从输入到状态的权重矩阵，U 是状态到状态的**转移矩阵**，两者在所有时刻共享。若还需要输出，再加一层变换：

$$\mathbf{y}_t = g\left(\mathbf{V}\mathbf{h}_t\right)$$

V 是状态到输出的权重矩阵。

4.2 展开后的视野

表面上看，h_t 只依赖于 h*{t−1}。但 h*{t−1} 又依赖 h*{t−2}，*h*{t−2} 又依赖 h_{t−3}，如此回溯，h_t 实际上与之前**所有**时刻的状态都有关系。

这正是“记忆”的数学含义：当前状态是整个历史序列的函数。序列越长，理论上它能记住的上下文越久远。

不过这只是理论上的可达性。实际上，早期信息要经过一长串变换才能影响到当前状态，很容易在传递中被稀释——这一点在第五节会看到。

五、训练：随时间反向传播

5.1 BPTT

对循环网络的训练，就是根据输出与真实结果之间的误差不断调整 W、U、V，直到达到预设要求。其目的可以概括为：让网络对输入序列做出正确的划分或预测。

训练方法仍然是基于梯度的反向传播，但与前馈网络不同的是，这里的反向传播是**沿时间**进行的（backpropagation through time）。由于每个状态都与之前的状态相关，对当前时刻的参数求偏导时，必然牵涉到前一时刻的参数，于是链式法则中多出了一组关于时间的中间变量。

具体做法是在展开后的计算图上做标准反向传播：把各时刻的损失加总，梯度沿时间轴逐级回传并累加。因为参数是共享的，同一个 W 在所有时刻都会收到一份梯度，最终更新的是这些梯度之和。

5.2 梯度沿时间衰减

展开后的网络有多深，取决于序列有多长。一条长度为 1000 的序列，展开后就是 1000 层。前面在 028 中讨论过的梯度弥散问题，在这里以更尖锐的形式出现：

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^{t} \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}}$$

从时刻 k 到时刻 t，梯度要连乘 t−k 个雅可比矩阵。每个矩阵的谱范数若小于 1，连乘结果就指数衰减；若大于 1，则指数爆炸。

后果很直接：网络“记得住”最近几十步，却学不到几百步之前的依赖。这正是 032 要解决的核心问题。

补充说明：梯度爆炸比梯度弥散更容易被发现，也更粗暴——损失突然变成 NaN，参数一步跳到无穷远。工程上的对策是梯度裁剪：先算梯度的范数，若超过阈值就按比例缩回去。梯度弥散则隐蔽得多，表现为模型能训、损失也在降，只是长距离依赖始终学不到。诊断时可以看不同时间跨度的误差曲线：若预测下一刻很准、预测几十步后就退化成平凡基线，多半是弥散。

六、结构扩展

6.1 双向循环网络

普通的循环网络，记忆只涉及过去的状态。若想让网络也利用来自未来的信息，就要让当前状态与之后的状态同样建立联系，得到的就是**双向循环神经网络**（bidirectional recurrent neural network）。

它包含正向与反向两个计算环节：正向计算中，时刻 t 的状态与 h*{t−1} 相关；反向计算中，同一时刻的状态与 h*{t+1} 相关。

两套计算的权重并不共享，因此需要分别计算正向和反向的结果，再把两者拼接或相加，作为该时刻隐藏层的最终表示。

这个设计在自然语言处理中几乎是标配：判断一个词在句中的含义，既要知道它前面说了什么，也要知道后面说了什么。完形填空正是最典型的场景——空格后面的内容，往往才是确定答案的关键。

6.2 深度循环网络

把深度结构引入双向循环网络，就得到**深度循环网络**。与其他深度结构一样，它具有多个隐藏层。

此时每个隐藏层的状态 h_t^(i) 既取决于同一时刻前一隐藏层的状态 h_t^(i−1)，也取决于同一隐藏层在前一时刻的状态 h_{t−1}^(i)。也就是说，信息同时在两个方向上流动：**沿时间**和**沿深度**。

深度结构的作用在于建立更清晰的表示。仍用完形填空类比：有些空只需看它所在的句子就能填，这对应单个隐藏层在时间维度上的依赖；有些空则要通读整段甚至全文，这对应深度结构在时间与空间两个维度上共有的依赖。

> **新手提示：双向不等于能用于在线预测**
>
> 双向循环网络在计算时刻 t 的输出时用到了 t 之后的信息，因此它只能用于“整段输入已经拿到”的任务，比如给一句话做标注、给一段音频做转写。对于真正的在线预测——每一刻都必须立刻给出结果——未来输入还不存在，双向结构无法使用，只能用单向网络。选型时先问一句：我的数据是离线的还是流式的。

七、递归神经网络

7.1 树状展开

循环网络的特点是在时间维度上共享参数，沿时间轴展开处理序列。如果换一种展开方式，把序列数据组织成树状结构，用到的就是**递归神经网络**。

递归网络使用的同样是共享权重，但这里的“相同”不体现在时间上，而体现在结构上：先把输入数据转化为某种拓扑结构，再在这个结构上递归地复用同一组权重，通过遍历得到结构化的预测。

7.2 语义向量

在自然语言处理中，递归网络能解决单纯词序列无法解决的问题。

比如“两个大学的老师”是个有歧义的短语：如果只把它拆成词序列，无论怎么排列都表达不出歧义所在；只有用语法解析树来表示，才能说清定语“两个”修饰的到底是“大学”还是“老师”。

把数据表示成树之后，递归网络的作用是将它进一步表示成向量，映射到语义空间中。在语义空间里，既可以度量单个向量的性质，比如判断一句话的感情色彩是褒是贬；也可以度量不同向量之间的关系，比如计算两个句子的语义相似度。

递归网络通过树结构把一个完整句子拆成若干分量的组合，最终生成的向量对应树的根节点。训练时同样用误差反向传播，误差从根节点一路传回每个叶子节点。

7.3 为什么不如循环网络流行

实际应用中，递归网络远不如循环网络受欢迎。

根本原因在于它对人工干预的依赖太重：要让训练集的每个句子都标注成语法解析树，不仅费时费力，而且标注质量直接决定模型上限。相比之下，循环网络只需要原始序列，不必提供结构信息。

从性价比看，递归网络在多数任务上并不划算。但它的思想并未消失——后来兴起的图神经网络，可以看作把“在树上递归”推广到“在任意图上传递消息”，其参数共享的思路与递归网络一脉相承。

四个常见误区

- **“RNN 记住了全部历史”**——隐藏状态是一个定长向量，容量有限。序列一长，早期信息会被后来者覆盖，这也是 LSTM、注意力机制出现的直接原因。
- **“双向 RNN 哪里都能用”**——它需要看到完整序列才能计算，因此不能用于实时流式预测；在线场景只能用单向结构。
- **“梯度消失只发生在深度方向”**——时间方向上的连乘同样会造成梯度消失或爆炸。序列越长，这个问题越严重。
- **“参数共享意味着模型简单”**——共享减少的是参数数量，不是计算图长度；沿时间展开后的图可能非常深，训练代价随序列长度线性增长。

八、要点回顾

1. 循环神经网络引入时间维度，适用于处理具有先后顺序的序列数据。
2. 它把变长输入切分成等长小块，用同一组权重逐块处理，参数量与序列长度无关。
3. 时间维度的参数共享，使网络能够利用数据之间的时域关联性。
4. 反馈结构带来记忆：当前状态是整个历史序列的函数。
5. 前馈网络的记忆是冻结的，只存在于训练好的参数中；循环网络的记忆随输入而变。
6. 状态更新为 h_t = f(Wx_t + Uh_{t−1})，W 是输入权重，U 是状态转移权重。
7. 训练采用随时间反向传播，在展开的计算图上做标准反向传播并累加梯度。
8. 梯度沿时间连乘，谱范数小于 1 时弥散、大于 1 时爆炸，长距离依赖难以学到。
9. 双向循环网络分别做正向与反向计算，把两套结果拼接，可利用未来信息。
10. 深度循环网络让信息同时沿时间和沿深度流动，建立更分层的表示。
11. 递归神经网络在树结构上共享权重，能刻画层次化语义，但依赖人工标注解析树。
12. 循环网络可视为递归网络的特例，递归网络可视为循环网络的推广。

九、名词汇总

名称	英文全称	英文缩写	简短介绍
循环神经网络	Recurrent Neural Network	RNN	在时间维度共享参数、具有记忆的序列模型
递归神经网络	Recursive Neural Network	RNN	在树结构上递归共享权重的网络
时间步	Time Step	—	序列中一个位置的离散时刻
隐藏状态	Hidden State	—	网络内部的记忆向量，随输入逐时刻更新
状态转移矩阵	Transition Matrix	—	把前一时刻状态映射到当前状态的权重矩阵
参数共享	Parameter Sharing	—	同一组权重在所有时间步重复使用
时域关联	Temporal Correlation	—	序列中不同时刻数据之间的依赖关系
随时间反向传播	Backpropagation Through Time	BPTT	在展开的计算图上沿时间轴反向求梯度
梯度裁剪	Gradient Clipping	—	梯度范数超阈值时按比例缩小以防爆炸
双向循环神经网络	Bidirectional RNN	BiRNN	同时做正向与反向计算、可利用未来信息的网络
深度循环网络	Deep RNN	—	多层堆叠、信息沿时间与深度同时流动的网络
完形填空	Cloze Test	—	根据上下文补全缺失词语的填空任务
语法解析树	Parse Tree	—	表示句子句法结构的树形表示
语义空间	Semantic Space	—	把语言片段映射为向量后所处的空间
图神经网络	Graph Neural Network	GNN	在任意图结构上传递并聚合消息的网络
序列建模	Sequence Modeling	—	对有序数据建立概率或映射模型的任务